Name - Mayank Heda
Rol no. - 2401035 Batch-T12
Subject - ASD&D

EXPERIMENT NO. 07

Aim: To implement monitoring using Prometheus / Grafana / Nagios

LO Mapping: LO5

LO5: Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.

Theory:

Continuous monitoring is a process to detect, report, respond all the attacks which occur in its infrastructure. Once the application is deployed into the server, the role of continuous monitoring comes in to play. The entire process is all about taking care of the company's infrastructure and respond appropriately.

Why We Need Nagios tool?

Here, are the important reasons to use Nagios monitoring tool:

- Detects all types of network or server issues
- Helps you to find the root cause of the problem which allows you to get the permanent solution to the problem
- Active monitoring of your entire infrastructure and business processes
- Allows you to monitor and troubleshoot server performance issues
- Helps you to plan for infrastructure upgrades before outdated systems create failures
- You can maintain the security and availability of the service
- Automatically fix problems in a panic situation
- Nagios is the most popular, open source, powerful monitoring system for any kind of infrastructure. It enables organizations to identify and resolve IT infrastructure problems before they affect critical business processes. Nagios has the capability of monitoring applications, services and entire IT infrastructure.
- NRPE is known as Nagios Remote Plugin Executor. It is designed to execute plugins on remote systems. On Windows hosts, the equivalent remote agent is NSClient++, which listens for Nagios requests and executes local checks (disk, CPU, memory, services, etc.).

- NSClient++ runs as a Windows service on the remote host and waits for requests from the Nagios server (usually over the NRPE protocol, which NSClient++ also supports). When the Nagios server needs to check a resource on that host, it sends a command; NSClient++ executes the check locally and returns the result to Nagios.

Note on running Nagios on Windows

Nagios Core is built for Linux and does not install natively on Windows. The standard practice for a Windows machine is to run the Nagios Core server inside a Docker container (using Docker Desktop with the WSL2 backend), and to monitor Windows hosts using NSClient++, which is the Windows counterpart of NRPE. All commands below are Windows PowerShell / Command Prompt commands.

1 - Pre-requisite: Install Docker Desktop

Download and install Docker Desktop for Windows and enable the WSL2 based engine. Verify the installation from PowerShell:

PS C:\Users\vishal> docker –version

```
C:\Users\Mayank Heda>docker --versic
Docker version 28.5.1, build e180ab8
```

PS C:\Users\vishal> docker version

```
C:\Users\Mayank Heda>docker version
Client:
 Version:           28.5.1
 API version:       1.51
 Go version:        go1.24.8
 Git commit:        e180ab8
 Built:             Wed Oct  8 12:19:16 2025
 OS/Arch:           windows/amd64
 Context:           desktop-linux
error during connect: Get "http://%2F%2F.%2Fpipe%2FdockerDe
LinuxEngine/v1.51/version": open //./pipe/dockerDesktopLinu
ne: The system cannot find the file specified.
```

Also make sure the WSL2 feature is enabled:

PS C:\Users\vishal> wsl –install

```
C:\Users\Mayank Heda>wsl --install
Downloading: Ubuntu
Installing: Ubuntu
Distribution successfully installed. It can be launched via
.exe -d Ubuntu'
Launching Ubuntu...
Provisioning the new WSL instance Ubuntu
This might take a while...
Create a default Unix user account: mayankheda
New password:
Retype new password:
passwd: password updated successfully
usermod: no changes
Help improve Ubuntu!
```

PS C:\Users\vishal> wsl --set-default-version 2

```
C:\Users\Mayank Heda>wsl --set-default-version 2
For information on key differences with WSL 2 please visit htt
://aka.ms/wsl2
The operation completed successfully.
```

2 - Pull and Run the Nagios Core Docker Image

Pull a ready-made Nagios Core image and run it as a container, mapping the web port to the host.

PS C:\Users\vishal> docker pull jasonrivers/nagios:latest

```
C:\Users\Mayank Heda>docker pull jasonrivers/nagios:lates
latest: Pulling from jasonrivers/nagios
a0372Oc2c16b: Pull complete
1b9771f55a12: Pull complete
dc3a88O12a3e: Pull complete
9288c4992679: Pull complete
cb259a83ac3d: Pull complete
fdbaa15b8986: Pull complete
a80db686d641: Pull complete
254354b7OacO: Pull complete
7a7447982198: Pull complete
37f75c8146e2: Pull complete
e4dc1d4eO1Oc: Pull complete
6fe99ef1bed3: Pull complete
Od59a81a56O8: Pull complete
e68fcda12cOc: Pull complete
dOO3d43O3177: Pull complete
5Oea27Od34e9: Pull complete
59215b8e47cf: Pull complete
c42O73aaf047: Pull complete
8Ob6ab7aef5d: Pull complete
748b632e75e7: Pull complete
c57O1b5e154O: Pull complete
f47c3af6bf44: Pull complete
a6b9Ob5bbb15: Pull complete
f2f9a1b3ce48: Pull complete
54b9e993e315: Pull complete
7edde1931fd2: Pull complete
a7bd6dce6a77: Pull complete
29O8452ff193: Pull complete
1413dee23aae: Pull complete
Digest: sha256:fee9cd6a2933e79Odde613d7Oc3a6O3f96978d65fa
17fce8O7eOOf2e2
Status: Downloaded newer image for jasonrivers/nagios:lat
docker.io/jasonrivers/nagios:latest
```

PS C:\Users\vishal> docker run --name nagios4 -d -p 8080:80 jasonrivers/nagios:latest (since 8080 is already occupied I used 8081)

```
C:\Users\Mayank Heda>docker run --name nagios4 -d -p 8081:80 jasonrivers/nagios:latest
17d03b8e70bc3c0b79ec517e6aca6c1ef689966C1eff83890933decf6d73d898o3d3316f79
```

Verify the container is up and running:

PS C:\Users\vishal> docker ps

```
C:\Users\Mayank Heda>docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED          STATUS         PORTS
      NAMES
17d03b8e70bc   jasonrivers/nagios:latest   "/usr/local/bin/star…"   10 seconds ago   Up 9 seconds   0.0.0.0:8081->80/tcp, [::]:8081->
80/tcp   nagios4
```

3 - Set the Apache/Nagios Admin Password

Set (or reset) the password for the nagiosadmin user inside the running container:

PS C:\Users\vishal> docker exec -it nagios4 htpasswd -c /opt/nagios/etc/htpasswd.users nagiosadmin

```
C:\Users\Mayank Heda>docker exec -it nagios4 htpasswd -c /opt/nagios/etc/htpasswd.users nagiosadmin
New password:
Re-type new password:
Adding password for user nagiosadmin
```

Enter and confirm the password when prompted (e.g. 123456).

4 - Access Nagios Web Interface

```
PS C:\Users\Mayank Heda\Downloads> curl.exe -L "https://github.com/mickem/nscp/releases/latest/download/NSCP-0.19.0-x64.msi" -o "$env:USERPROFILE\Downloads\NSCP.msi"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time   Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0                              0
  0     0    0     0    0     0      0      0                              0
100 79.03M 100 79.03M    0     0  8.07M      0   00:09   00:09          7.90M
PS C:\Users\Mayank Heda\Downloads> Get-Item "$env:USERPROFILE\Downloads\NSCP.msi" | Select-Object Name,Length

Name       Length
----       ------
NSCP.msi 82874368


PS C:\Users\Mayank Heda\Downloads> msiexec /i "$env:USERPROFILE\Downloads\NSCP.msi"
PS C:\Users\Mayank Heda\Downloads> Get-Service nscp

Status   Name               DisplayName
------   ----               -----------
Running  nscp               NSClient++ Monitoring Agent
```

Access the Nagios web interface from a browser on the Windows host:

http://localhost:8080/nagios/

Prompting for Authentication Password -

username: nagiosadmin

Password: 123456 (which you entered while configuration)

5 - Install NSClient++ on the Windows Remote Host

On the Windows machine you want to monitor, download and silently install NSClient++ (the Windows equivalent of the NRPE agent):

PS C:\Users\vishal> Invoke-WebRequest -Uri "https://github.com/mickem/nscp/releases/latest/download/NSCP-0.5.2-x64.msi" -OutFile "NSCP.msi"

```
PS C:\Windows\system32> curl.exe -L "https://github.com/mickem/nscp/releases/latest/download/NSCP-0.5.2-x64.msi" -o "$env:USERPROFILE\Downloads\NSCP.msi"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time   Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0                              0
100     9  100     9    0     0     11      0                              0
PS C:\Windows\system32>
```

PS C:\Users\vishal> Start-Process msiexec.exe -ArgumentList '/i NSCP.msi ADDLOCAL=ALL MODULES=CheckExternalScripts,NRPEServer /qn' -Wait

6 - Configure NSClient++

Edit the NSClient++ configuration file and add the Nagios server's IP address to the allowed hosts list:

PS C:\Program Files\NSClient++> notepad nsclient.ini

allowed hosts=127.0.0.1,192.168.1.100

```
Windows IP Configuration


Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 2:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . : lan
   IPv6 Address. . . . . . . . . . . : 2409:40c0:11b6:78ca:5d2c:43b7:6fe8:e5a1
   Temporary IPv6 Address. . . . . . : 2409:40c0:11b6:78ca:2d3b:f93a:9979:796d
   Link-local IPv6 Address . . . . . : fe80::3b0b:3980:e790:f517%5
   IPv4 Address. . . . . . . . . . . : 192.168.31.230
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::7ebf:77ff:fe07:f211%5
                                       192.168.31.1

Ethernet adapter vEthernet (WSL (Hyper-V firewall)):

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::df9f:f125:1340:48d3%41
   IPv4 Address. . . . . . . . . . . : 172.29.112.1
   Subnet Mask . . . . . . . . . . . : 255.255.240.0
   Default Gateway . . . . . . . . . :
```

```
File    Edit    View

; Load on startup - If the module should be loaded on startup
CheckExternalScripts = enabled

; Load on startup - If the module should be loaded on startup
CheckNSCP = enabled


; in flight - TODO
[/settings/NRPE/server]

verify mode = peer-cert

tls version = tlsv1.2+

insecure = false


; in flight - TODO
[/settings/default]

allowed hosts = 127.0.0.1,192.168.31.230

password = t27TXTHnUHZrYiV9
```

Where 192.168.1.100 is your Nagios server (Docker host) IP address.

After saving the file, restart the NSClient++ service so the change takes effect:

PS C:\Users\vishal> Restart-Service nscp

```
PS C:\Users\Mayank Heda\Downloads> Restart-Service nscp
PS C:\Users\Mayank Heda\Downloads> Get-Service nscp

Status   Name               DisplayName
------   ----               -----------
Running  nscp               NSClient++ Monitoring Agent
```

7 - Verify Connectivity from Nagios

From the Nagios server (inside the Docker container), run the NRPE check against the Windows remote host to confirm connectivity. Here 192.168.1.50 is the remote Windows host's IP address.

PS C:\Users\vishal> docker exec -it nagios4 /opt/nagios/libexec/check_nrpe -H 192.168.1.50

NRPE v2.15

```
PS C:\Program Files\NSClient++> docker exec -it nagios4 /opt/nagios/libexec/check_nr
pe -H 192.168.31.230 -3
I (0.19.0 2026-09-07) seem to be doing fine...
```

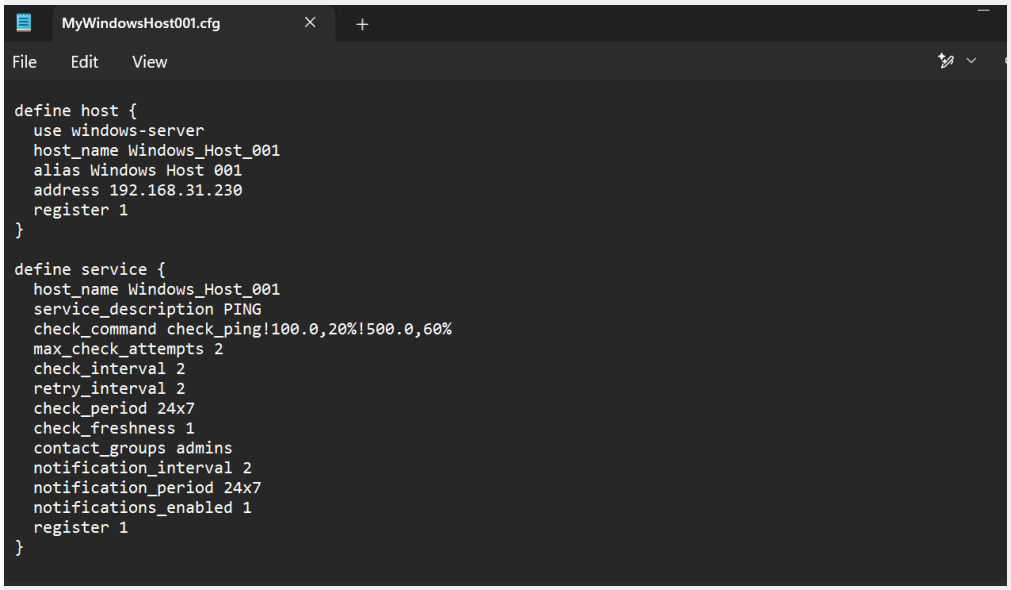
8 - Add the Windows Host in Nagios

Create a configuration file for the new host inside the Nagios container. We also define a service that continuously checks whether the host is reachable (PING check).

PS C:\Users\vishal> docker exec -it nagios4 sh -c "vi /opt/nagios/etc/servers/MyWindowsHost001.cfg"

```
define host {
   use             windows-server
   host_name       Windows_Host_001
   alias           Windows Host 001
   address         192.168.1.50
   register        1
}
define service{
   host_name             Windows_Host_001
   service_description   PING
   check_command         check_ping!100.0,20%!500.0,60%
   max_check_attempts    2
   check_interval        2
```

```
    retry_interval        2
    check_period          24x7
    check_freshness       1
    contact_groups        admins
    notification_interval 2
    notification_period   24x7
    notifications_enabled 1
    register              1
}
```



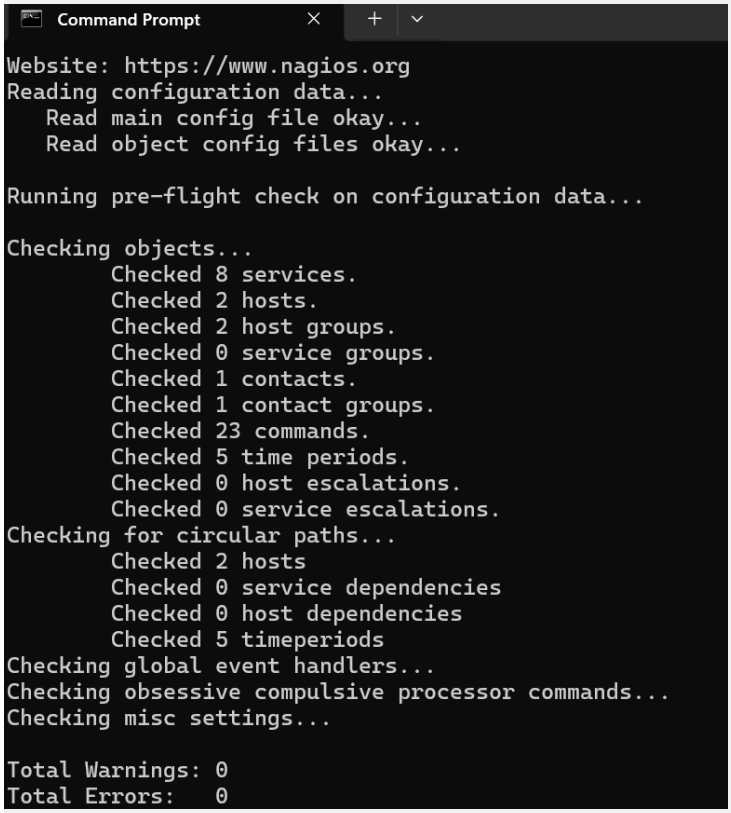

```
define host {
  use windows-server
  host_name Windows_Host_001
  alias Windows Host 001
  address 192.168.31.230
  register 1
}

define service {
  host_name Windows_Host_001
  service_description PING
  check_command check_ping!100.0,20%!500.0,60%
  max_check_attempts 2
  check_interval 2
  retry_interval 2
  check_period 24x7
  check_freshness 1
  contact_groups admins
  notification_interval 2
  notification_period 24x7
  notifications_enabled 1
  register 1
}
```

Verify the configuration file for errors, then restart the Nagios container so the changes take effect:

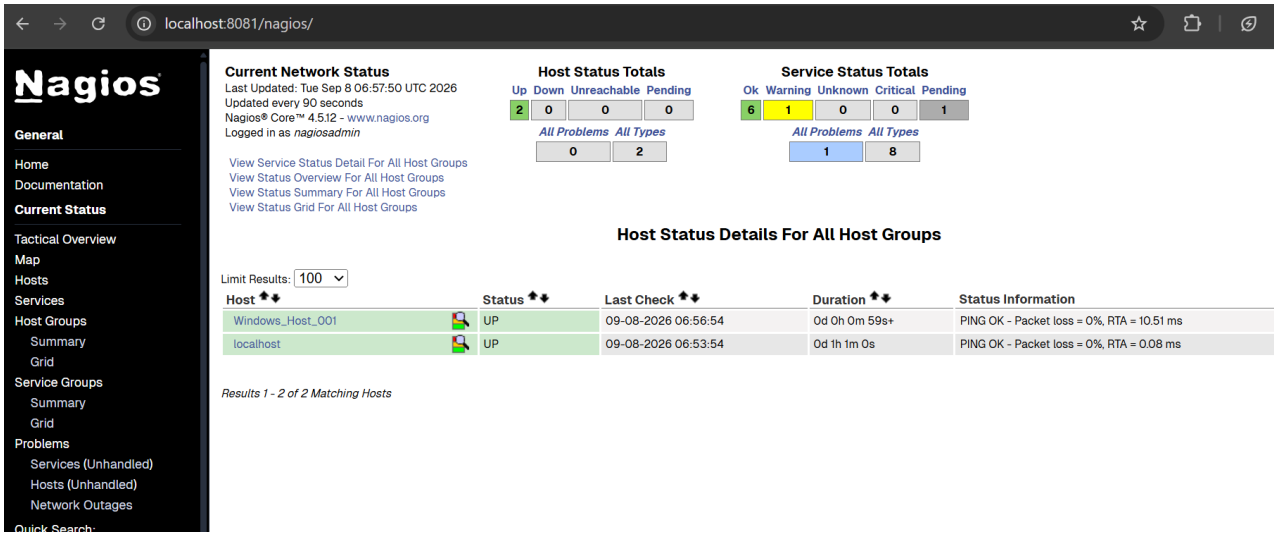
PS C:\Users\vishal> docker exec -it nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg

PS C:\Users\vishal> docker restart nagios4



9 - Check Host in Nagios Web Interface

Open the Nagios web interface (http://localhost:8080/nagios/) and check that the new Windows host has been added and is being monitored under Current Status -> Hosts.



Current Network Status
Last Updated: Tue Sep 8 06:58:57 UTC 2026
Updated every 90 seconds
Nagios® Core™ 4.5.12 - www.nagios.org
Logged in as *nagiosadmin*

View History For all hosts
View Notifications For All Hosts
View Host Status Detail For All Hosts

Host Status Totals

Up	Down	Unreachable	Pending
2	0	0	0

All Problems	All Types
0	2

Service Status Totals

Ok	Warning	Unknown	Critical	Pending
7	1	0	0	0

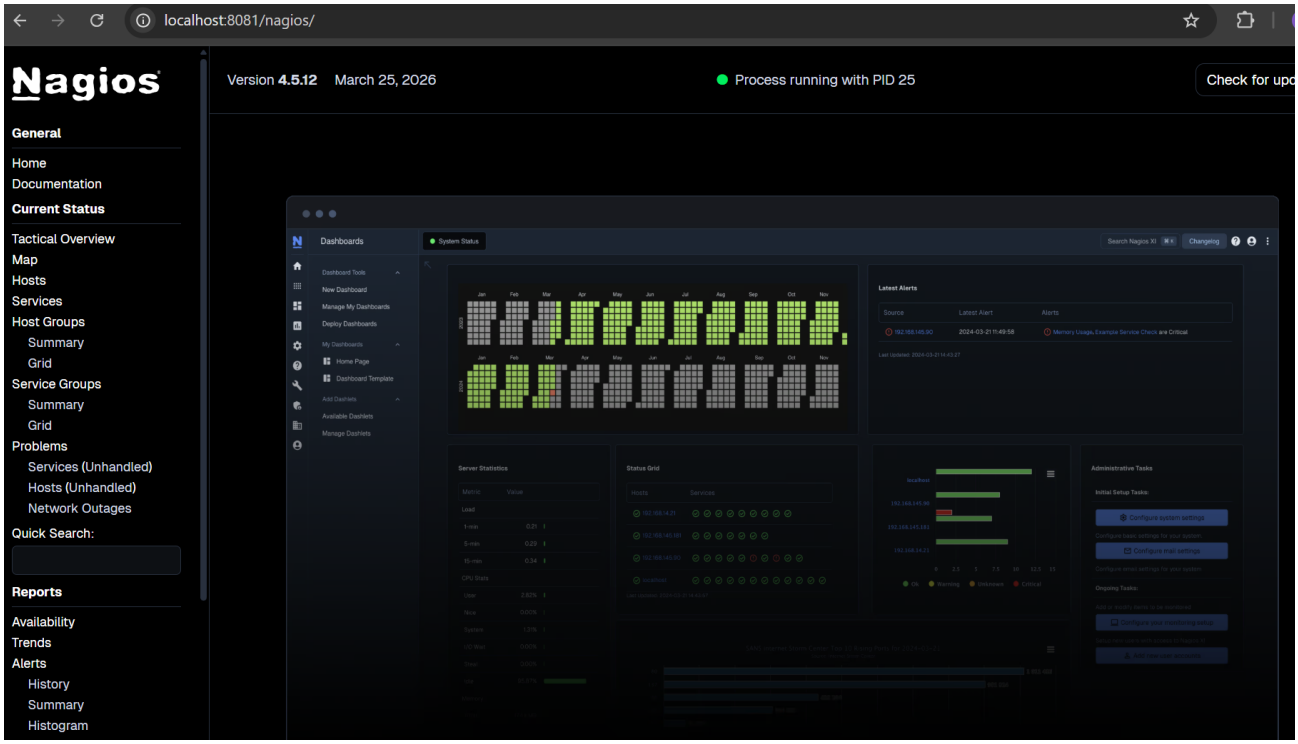
All Problems	All Types
1	8

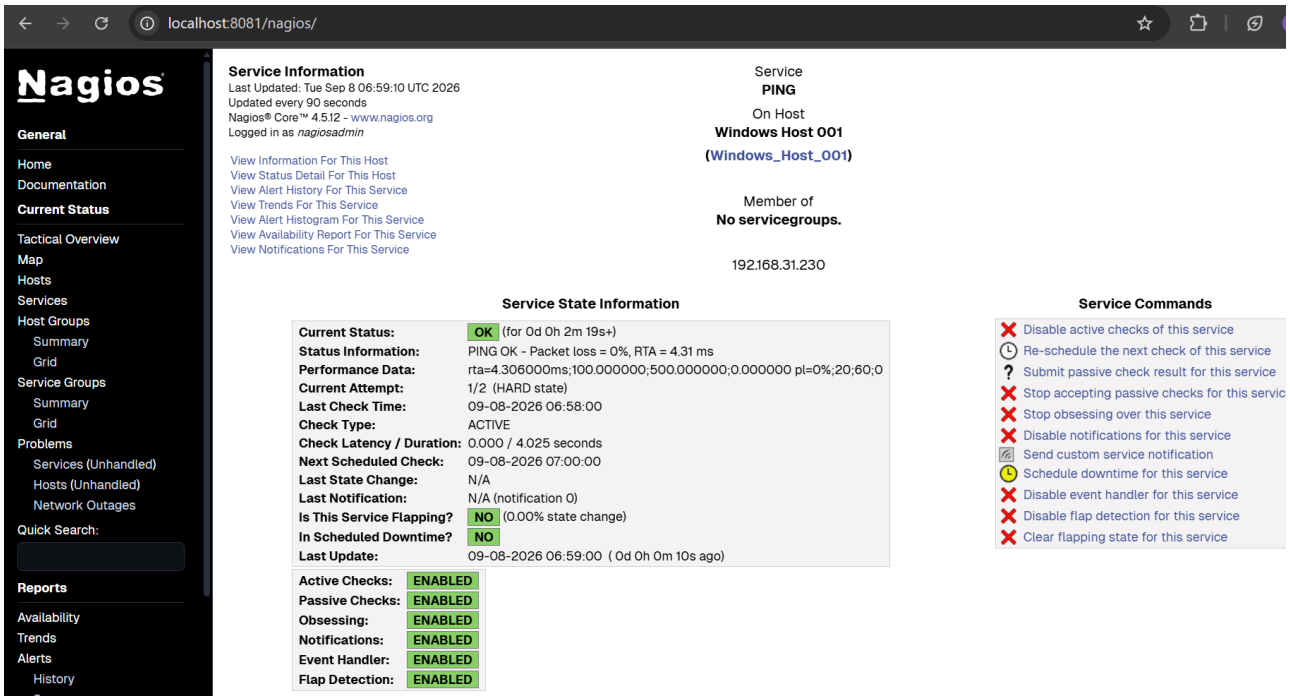
Service Status Details For All Hosts

Limit Results: 100

Host	Service	Status	Last Check	Duration	Attempt	Status Information
Windows_Host_001	PING	OK	09-08-2026 06:58:00	0d 0h 2m 6s+	1/2	PING OK - Packet loss = 0%, RTA = 4.31 ms
localhost	Current Load	OK	09-08-2026 06:57:28	0d 1h 1m 29s	1/4	OK - load average: 0.13, 0.06, 0.07
	Current Users	OK	09-08-2026 06:58:11	0d 1h 0m 46s	1/4	USERS OK - 0 users currently logged in
	HTTP	WARNING	09-08-2026 06:56:54	0d 0h 57m 3s	4/4	HTTP WARNING: HTTP/1.1 401 Unauthorized - 695 bytes in 0.002 second response time
	PING	OK	09-08-2026 06:54:37	0d 0h 59m 20s	1/4	PING OK - Packet loss = 0%, RTA = 0.07 ms
	Root Partition	OK	09-08-2026 06:55:20	0d 0h 58m 37s	1/4	DISK OK - free space: / 973561 MiB (99.48% inode=100%):
	Swap Usage	OK	09-08-2026 06:56:03	0d 0h 57m 54s	1/4	SWAP OK - 100% free (2048 MB out of 2048 MB)
	Total Processes	OK	09-08-2026 06:56:46	0d 0h 57m 11s	1/4	PROCS OK: 44 processes with STATE = RSZDT

Results 1 - 8 of 8 Matching Services





Conclusion:

Nagios Core was successfully implemented for continuous monitoring. The Windows host was added successfully, and its **PING service was monitored with an OK status** through the Nagios web interface.